



BIRZEIT UNIVERSITY

Faculty of Engineering and Technology

Department of Computer Science

Academic Year 2025/2026 Summer Semester

ARTIFICIAL INTELLIGENCE

COMP338

Course Project

AI-Based University Course Scheduling Using Genetic Algorithm

Prepared by:

Ramz Abufarha

1230963

Dr. Abdelrahman Hamarsha

Section 1

6/9/2026

Table of Contents

1.	Introduction	4
2.	Problem Formulation	5
2.1	Problem Description	5
2.2	Input Data	5
2.3	Chromosome Representation	6
2.4	Hard Constraints	8
2.5	Soft Constraints	9
2.6	Fitness Function	10
3.	Genetic Algorithm	11
3.1	Overview	11
3.2	Population Initialization	12
3.3	Fitness Evaluation	13
3.4	Tournament Selection	14
3.5	Two-Point Crossover	14
3.6	Random Reassignment Mutation	15
3.7	Room Repair	16
3.8	Elitism	16
3.9	Generation Replacement	17
3.10	Termination	17
3.11	Best Solution	17
4.	Code Explanation	18
4.1	Project Structure	18
4.2	Data Model Classes	19
4.2.1	Course	19
4.2.2	Instructor	19
4.2.3	Room	19
4.2.4	TimeSlot	20
4.2.5	StudentGroup and CourseGroup	20
4.3	Gene and Chromosome	21
4.3.1	Gene	21
4.3.2	Chromosome	22

4.4 Population Initialization	23
4.5 Fitness Function	23
4.6 Selection	25
4.6.1 Tournament Selection	25
4.7 Crossover	26
4.7.1 Two-Point Crossover	26
4.8 Mutation	27
4.8.1 Random Reassignment Mutation	27
4.9 Constraint Evaluation	29
4.9.1 Hard Constraint Evaluation	29
4.9.2 Soft Constraint Evaluation	29
4.9.3 Schedule Evaluation	30
4.10 Genetic Algorithm Implementation	31
4.11 Convergence Graph	32
5. Experimental Work	33
5.1 Experimental Setup	33
5.2 Baseline Run	34
5.3 Convergence Analysis	35
5.4 Population Size Experiment	36
5.5 Mutation Rate Experiment	37
5.6 Crossover Rate Experiment	38
5.7 Dataset Size Experiment	39
5.8 Overall Experimental Results	40
5.9 Discussion of Results	41
6 Conclusion	42

1. Introduction

University course scheduling is a complex problem that requires assigning courses to available time slots and rooms while satisfying several constraints. A valid timetable must avoid conflicts between instructors and rooms, provide sufficient room capacity, respect room types, consider instructor availability, and prevent conflicts between courses attended by the same group of students.

Traditional approaches to course scheduling can become difficult as the number of courses, instructors, rooms, and constraints increases. Therefore, this project investigates the use of a **Genetic Algorithm (GA)** as an intelligent optimization approach for generating university course timetables.

In this project, each candidate solution represents a complete timetable, where each course is assigned a specific day, time slot, and room. The Genetic Algorithm evolves a population of candidate timetables through selection, crossover, mutation, and elitism. Each timetable is evaluated using a fitness function based on the number of hard and soft constraint violations.

The primary objective is to obtain a timetable with **zero hard constraint violations** while minimizing soft constraint violations. The project also investigates the effect of different population sizes, mutation rates, crossover rates, and dataset sizes on the performance of the Genetic Algorithm.

The implemented system was developed in Java and includes the complete Genetic Algorithm process, constraint evaluation, timetable generation, convergence analysis, and experimental evaluation.

2. Problem Formulation

2.1 Problem Description

The scheduling problem is formulated as an optimization problem in which the Genetic Algorithm searches for a high-quality timetable among a large number of possible scheduling combinations.

For each course, the system must determine:

- **Day**
- **Time slot**
- **Room**

The project dataset contains **10 courses, 8 instructors, 6 rooms, 5 working days, and 4 time periods per day**. Therefore, there are 20 available time slots across the week and 120 possible scheduling positions when the six rooms are considered.

The project also requires the algorithm to satisfy hard constraints and minimize undesirable scheduling patterns represented by soft constraints

2.2 Input Data

The original dataset contains the following courses:

ID	Course	Students	Instructor	Type
C01	AI	35	Dr. Ahmad	Lecture
C02	Database	40	Dr. Sara	Lecture
C03	Networks	30	Dr. Omar	Lecture
C04	Operating Systems	35	Dr. Ahmad	Lecture
C05	Software Engineering	45	Dr. Lina	Lecture
C06	Algorithms	40	Dr. Khaled	Lecture
C07	Data Mining	25	Dr. Noor	Lecture
C08	Computer Graphics	30	Dr. Rami	Lab
C09	Web Programming	25	Dr. Huda	Lab
C10	Machine Learning	35	Dr. Noor	Lecture

The available rooms are:

Room	Capacity	Type
R101	50	Lecture
R102	40	Lecture
R103	35	Lecture
R104	25	Lecture
LAB1	30	Lab
LAB2	25	Lab

The available days are **Sunday through Thursday**, with four time periods:

Period	Time
1	08:00–09:00
2	09:00–10:00
3	10:00–11:00
4	11:00–12:00

This gives:

5 days × 4 periods = 20 time slots.

2.3 Chromosome Representation

A **chromosome represents one complete university timetable**. Each chromosome contains one gene for every course.

Each gene represents the scheduling assignment of one course:

$$\text{Gene} = (\text{Day}, \text{Time}, \text{Room})$$

Therefore:

$$\text{Chromosome} = [\text{Gene}_1, \text{Gene}_2, \dots, \text{Gene}_n]$$

where n is the number of courses.

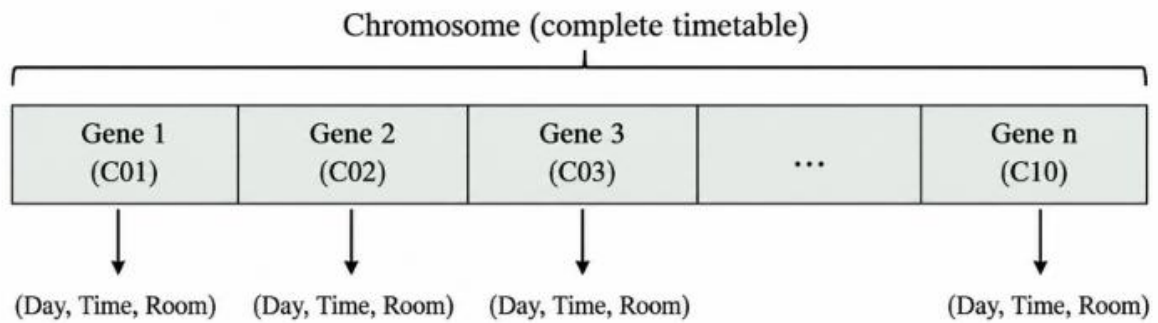


Figure 1 Chromosome representation used in the scheduling system

For example, a gene could represent:

AI → Sunday, 10:00–11:00, R101

This means that the AI course is scheduled on Sunday from 10:00 to 11:00 in room R101.

The project specification explicitly defines the chromosome as a complete timetable and each gene as an assignment containing day, time, and room.

2.4 Hard Constraints

Hard constraints are constraints that should not be violated in a valid timetable. The implementation evaluates six hard constraints:

H1 – Instructor Conflict

An instructor cannot teach two courses at the same day and time.

H2 – Room Conflict

A room cannot be assigned to two courses at the same day and time.

H3 – Student Conflict

Two courses attended by the same student group cannot occur at the same day and time.

H4 – Room Capacity

The assigned room must have sufficient capacity for the number of students enrolled in the course.

H5 – Room Type

The room type must match the course type. For example, a Lab course must use a Lab room.

H6 – Instructor Availability

A course cannot be scheduled during a time when its instructor is unavailable.

These constraints correspond to the six hard constraints specified in the project requirements.

Table 1 Hard constraints implemented in the system

Constraint	Description
H1 – Instructor Conflict	Prevents an instructor from teaching two courses at the same time.
H2 – Room Conflict	Prevents a room from being assigned to two courses at the same time.
H3 – Student Conflict	Prevents courses of the same student group from overlapping.
H4 – Room Capacity	Ensures that room capacity is sufficient for the course.
H5 – Room Type	Ensures that the room type matches the course type.
H6 – Instructor Availability	Ensures that courses are scheduled when instructors are available.

2.5 Soft Constraints

Soft constraints do not necessarily make a timetable invalid, but violating them reduces its quality.

The implementation uses five soft constraints:

S1 – Avoid Early Slots

Penalize courses scheduled during the first period, 08:00–09:00.

S2 – Avoid Late Slots

Penalize courses scheduled during the last period, 11:00–12:00.

S3 – Minimize Student Gaps

Penalize undesirable gaps between courses belonging to the same student group.

S4 – Instructor Preferences

Penalize assignments that do not respect the defined morning preferences for selected instructors.

S5 – Minimize Working Days

Penalize schedules that cause a student group to have courses spread across all five working days.

The project requires at least three soft constraints, and the implementation provides five.

Table 2 Soft constraints implemented in the system

Constraint	Description
S1 – Avoid Early Slots	Penalizes courses scheduled at 08:00–09:00.
S2 – Avoid Late Slots	Penalizes courses scheduled at 11:00–12:00.
S3 – Student Gaps	Penalizes undesirable gaps between courses of the same student group.
S4 – Instructor Preferences	Penalizes assignments that do not follow defined instructor preferences.
S5 – Working Days	Penalizes schedules that spread a student group across all five working days.

2.6 Fitness Function

The Genetic Algorithm evaluates every chromosome using a penalty-based fitness function.

The total penalty is:

$$\text{Penalty} = W_H H + W_S S$$

where:

- H = number of hard constraint violations
- S = number of soft constraint violations
- $W_H = 100$
- $W_S = 10$

Therefore:

$$\text{Penalty} = 100H + 10S$$

The fitness is then calculated as:

$$\text{Fitness} = \frac{1}{1 + \text{Penalty}}$$

This gives hard constraint violations a much larger penalty than soft violations.

For a perfect timetable:

$$H = 0, S = 0$$

Therefore:

$$\text{Penalty} = 0$$

and:

$$\text{Fitness} = \frac{1}{1 + 0} = 1$$

3. Genetic Algorithm

3.1 Overview

A Genetic Algorithm (GA) is an optimization technique inspired by the process of natural selection. It maintains a population of candidate solutions and improves the population over multiple generations by selecting better solutions and applying genetic operators such as crossover and mutation.

In this project, each individual represents a complete university timetable. The quality of an individual is determined using a fitness function based on hard and soft constraint violations. Better timetables receive higher fitness values.

The implemented Genetic Algorithm follows the general process:

Initial Population → Fitness Evaluation → Parent Selection → Crossover → Mutation → Repair → Elitism → New Population → Termination

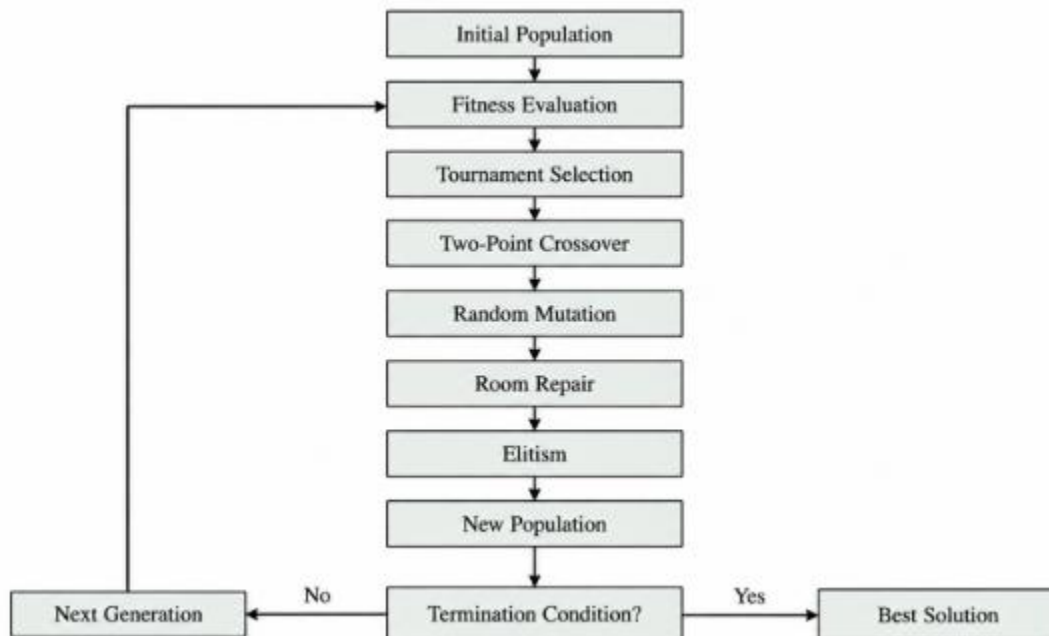


Figure 2 Workflow of the implemented Genetic Algorithm

This follows the Genetic Algorithm approach presented in the course material, where individuals are evaluated according to fitness and new generations are produced through selection, crossover, and mutation.

3.2 Population Initialization

The algorithm begins by generating an initial population of candidate timetables.

In the implementation, the population size is configurable. The default project configuration uses:

Parameter	Value
Population Size	100
Maximum Generations	500
Crossover Rate	80%
Mutation Rate	5%
Elitism	5%
Tournament Size	3

Each chromosome contains one gene for every course. For every gene, the system randomly selects a time slot and a room.

Therefore, the initial population consists of different randomly generated timetables that are later evaluated and improved by the Genetic Algorithm.

The default population size, generation limit, crossover rate, mutation rate, and elitism configuration follow the project specification.

3.3 Fitness Evaluation

After generating the population, every chromosome is evaluated.

The evaluation process calculates:

1. The number of hard constraint violations.
2. The number of soft constraint violations.
3. The total penalty.
4. The fitness value.

The penalty function is:

$$\text{Penalty} = 100H + 10S$$

where H is the number of hard violations and S is the number of soft violations.

The fitness is:

$$\text{Fitness} = \frac{1}{1 + \text{Penalty}}$$

Therefore, a timetable with fewer violations receives a higher fitness value.

For example, if:

$$H = 0, S = 0$$

then:

$$\text{Penalty} = 0$$

and:

$$\text{Fitness} = 1$$

Thus, **1.0 is the optimal fitness value for the implemented fitness function.**

3.4 Tournament Selection

The implementation uses **Tournament Selection** to select parents for reproduction.

In tournament selection, a small number of individuals are randomly chosen from the population. Their fitness values are compared, and the individual with the highest fitness is selected as a parent.

In this implementation, the tournament size is:

Tournament Size = 3

The process is:

1. Randomly select three chromosomes.
2. Calculate their fitness values.
3. Compare their fitness values.
4. Select the chromosome with the highest fitness.
5. Repeat the process when another parent is required.

This allows stronger solutions to have a higher probability of being selected while still maintaining some randomness in the selection process.

3.5 Two-Point Crossover

The implemented crossover operator is **Two-Point Crossover**.

Two crossover points are randomly selected within the chromosome. The genes between these two points are exchanged between the two parent chromosomes to create two offspring.

For example:

Parent 1:

[G1 G2 | G3 G4 G5 | G6 G7 G8]

Parent 2:

[H1 H2 | H3 H4 H5 | H6 H7 H8]

After crossover:

Child 1:

[G1 G2 | H3 H4 H5 | G6 G7 G8]

Child 2:

[H1 H2 | G3 G4 G5 | H6 H7 H8]

This allows offspring to inherit parts of the timetable from both parents.

The crossover rate determines whether crossover is applied to a selected pair. The default value in the project is **80%**. The course material describes crossover as exchanging parts of parent representations to produce new offspring.

3.6 Random Reassignment Mutation

Mutation introduces random changes into offspring to maintain diversity within the population.

The implemented mutation operator randomly selects **one gene** from a chromosome and reassigns:

- a new random time slot
- a new random room

For example:

Before mutation: AI → Sunday, 10:00–11:00, R101

After mutation: AI → Tuesday, 09:00–10:00, R102

The mutation rate determines how frequently this operation is applied.

The default mutation rate is:

5%

The project specifically requires random reassignment mutation and experiments using mutation rates of 1%, 5%, 10%, and 20%.

3.7 Room Repair

After crossover or mutation, a chromosome may contain an unsuitable room assignment. For example, a course may be assigned to a room that is too small or has the wrong type.

To reduce these violations, the implementation uses a **Room Repair** step.

For each gene, the repair procedure searches for a room that:

- matches the course type,
- has enough capacity,
- is not already occupied at the same day and time by another repaired course.

If a suitable room is found, the assignment is replaced with that room.

This repair step helps the algorithm produce more feasible timetables and was particularly useful for the larger experimental datasets.

3.8 Elitism

Elitism is used to preserve the best solutions between generations.

In the implementation, the best 5% of the current population are copied directly into the next generation.

For the default population size:

$$\text{Population} = 100$$

the number of elite chromosomes is:

$$100 \times 0.05 = 5$$

Therefore, the five best chromosomes are preserved.

The purpose of elitism is to ensure that a high-quality solution is not lost because of crossover or mutation.

The project specifies an elitism rate of 5%.

3.9 Generation Replacement

After applying selection, crossover, mutation, room repair, and elitism, the new chromosomes form the next generation. The previous population is replaced by this new population while the elite chromosomes are preserved.

The process is repeated for successive generations. In each generation, the population is evaluated again, and the best solution found so far is retained.

3.10 Termination

The Genetic Algorithm continues generating new populations until a termination condition is reached.

The project specification defines a maximum of 500 generations and also considers termination when the solution stops improving for a specified number of generations. In the implementation, the algorithm terminates early if the best fitness does not improve for 50 consecutive generations.

Therefore, the algorithm stops either when the maximum number of generations is reached or when no improvement is observed for 50 consecutive generations. This helps avoid unnecessary computation after the population has converged.

3.11 Best Solution

At the end of the Genetic Algorithm, the chromosome with the highest fitness found during the run is considered the best solution.

The best chromosome represents a complete university timetable, where each gene contains the course, assigned day, time slot, and room.

The final solution is evaluated using the implemented hard and soft constraints. The system reports the number of scheduled courses, hard constraint violations, soft constraint violations, final penalty, best fitness, and number of generations used.

A fitness value of 1.0 represents the optimal value for the implemented fitness function, corresponding to zero hard and zero soft constraint violations.

4. Code Explanation

4.1 Project Structure

The project was implemented in Java using a modular structure that separates the data model, Genetic Algorithm components, constraint evaluation, and utility functions.

The main source code is organized into the following packages:

- **model:** Contains the classes that represent the main scheduling entities, including courses, instructors, rooms, time slots, genes, chromosomes, and student groups.
- **data:** Contains the classes responsible for creating the project dataset and the larger experimental datasets.
- **ga:** Contains the main Genetic Algorithm components, including population initialization, fitness evaluation, selection, crossover, mutation, elitism, and the main Genetic Algorithm process.
- **constraints:** Contains the hard and soft constraint evaluation and the room repair mechanism.
- **utils:** Contains supporting utility classes such as random number generation, time-slot generation, and convergence graph generation.

The project also contains main execution and testing classes. `Main` runs the scheduling system and displays the final timetable, while `ExperimentRunner` performs the required experiments. `ConstraintTest` is used to test the constraint-handling and room-repair process.

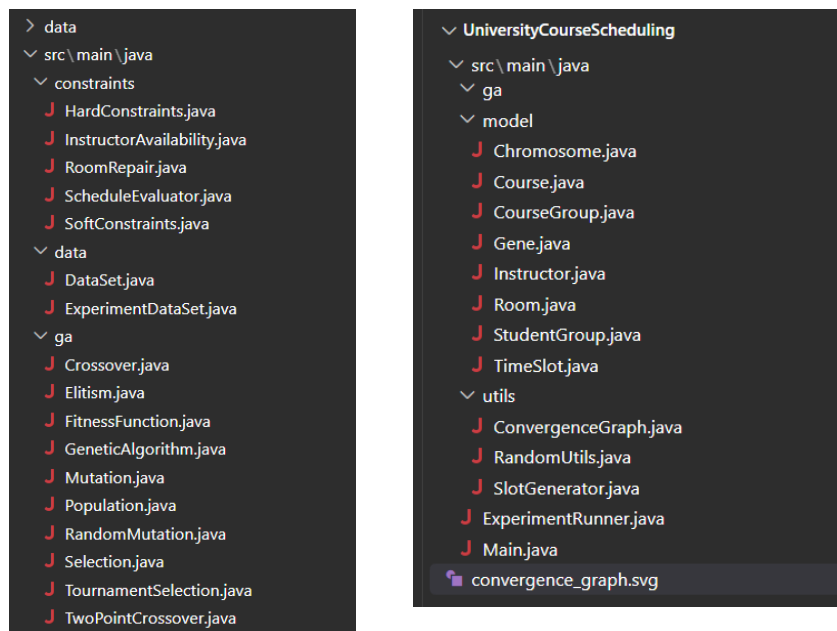


Figure 3 Project structure of the implemented university course scheduling system

4.2 Data Model Classes

The `model` package contains the classes used to represent the main entities of the university scheduling problem. These classes provide the data required by the Genetic Algorithm to construct and evaluate timetables.

The main data model classes are `Course`, `Instructor`, `Room`, `TimeSlot`, `Gene`, `Chromosome`, `StudentGroup`, and `CourseGroup`.

4.2.1 Course

The `Course` class represents a university course. It stores the course ID, course name, number of enrolled students, assigned instructor, and course type.

The main attributes are:

- `id` – uniquely identifies the course.
- `name` – stores the course name.
- `students` – stores the number of enrolled students.
- `instructor` – represents the instructor teaching the course.
- `type` – specifies whether the course is a Lecture or Lab.

These attributes are used when generating timetable assignments and evaluating constraints such as instructor conflicts, room capacity, and room type.

4.2.2 Instructor

The `Instructor` class represents an instructor in the scheduling system. It contains an instructor ID and name.

The instructor ID is also used by the availability mechanism to determine whether the instructor can teach at a particular time slot.

4.2.3 Room

The `Room` class represents an available classroom or laboratory. It stores:

- `id` – room identifier.
- `capacity` – maximum number of students.
- `type` – room type, such as Lecture or Lab.

This information is used to evaluate room capacity and room type constraints and during the room repair process.

4.2.4 TimeSlot

The `TimeSlot` class represents a scheduling time. It contains:

- `day` – the day of the week.
- `time` – the time period.

The class is used together with a room to determine where and when a course is scheduled.

4.2.5 StudentGroup and CourseGroup

The `StudentGroup` class represents a group of students using a group ID and name.

The `CourseGroup` class provides the relationship between a course and a student group. This relationship is used when evaluating the student conflict constraint, ensuring that courses attended by the same group are not scheduled at the same day and time.

4.3 Gene and Chromosome

The `Gene` and `Chromosome` classes implement the chromosome representation used by the Genetic Algorithm.

4.3.1 Gene

The `Gene` class represents the scheduling assignment of one course. It contains three main components:

- `course` – the course being scheduled.
- `timeSlot` – the assigned day and time.
- `room` – the assigned room.

The constructor receives these three components and stores them inside the gene. Getter methods are provided to access the course, time slot, and room.

Therefore, each gene represents one assignment in the form:

Gene=(*Course*, *Day*, *Time*, *Room*)

For example: AI → Sunday, 10:00–11:00, R101

This means that the gene represents the assignment of the AI course to the specified day, time, and room.

```
src > main > java > model > J Gene.java > Gene
1  package model;
2
3  public class Gene {
4
5      private Course course;
6      private TimeSlot timeSlot;
7      private Room room;
8
9      public Gene(Course course, TimeSlot timeSlot, Room room) {
10         this.course = course;
11         this.timeSlot = timeSlot;
12         this.room = room;
13     }
14
15     public Course getCourse() {
16         return course;
17     }
18
19     public TimeSlot getTimeSlot() {
20         return timeSlot;
21     }
22
23     public Room getRoom() {
24         return room;
25     }
26 }
```

Figure 4 implementation of the *Gene* class

4.3.2 Chromosome

The `Chromosome` class represents one complete timetable. It contains an array of `Gene` objects:

`Gene[]`.

Each position in the array represents one course assignment. Therefore, if the dataset contains 10 courses, a chromosome contains 10 genes.

The chromosome provides a constructor for storing the gene array and a getter method for accessing the genes.

The representation can therefore be summarized as:

$$\text{Chromosome} = [\text{Gene}_1, \text{Gene}_2, \dots, \text{Gene}_n]$$

where n is the number of courses.

This representation allows the Genetic Algorithm operators to manipulate complete timetables by working with their genes during selection, crossover, and mutation.

```
src > main > java > model > J Chromosome.java > Chromosome
1  package model;
2
3  public class Chromosome {
4
5      private Gene[] genes;
6
7      public Chromosome(Gene[] genes) {
8          this.genes = genes;
9      }
10
11     public Gene[] getGenes() {
12         return genes;
13     }
14 }
```

Figure 5 Implementation of the `Chromosome` class

4.4 Population Initialization

The `Population` class is responsible for generating the initial population of candidate timetables. Each chromosome in the population represents one complete timetable, and each chromosome contains one gene for every course.

The population size is configurable. In the default configuration, the Genetic Algorithm starts with a population of **100 chromosomes**.

For each chromosome, the implementation processes all courses and creates one gene for each course. For every gene, a time slot and a room are selected randomly from the available time slots and rooms.

The initialization process can be summarized as follows:

1. Create an empty chromosome.
2. Iterate through all courses in the dataset.
3. Randomly select a time slot.
4. Randomly select a room.
5. Create a gene containing the course, selected time slot, and room.
6. Add the gene to the chromosome.
7. Repeat the process until all courses are assigned.
8. Repeat the chromosome-generation process until the required population size is reached.

As a result, the initial population contains multiple different candidate timetables. These chromosomes are then evaluated using the fitness function and used as the starting point for the subsequent Genetic Algorithm operations.

4.5 Fitness Function

The `FitnessFunction` class is responsible for converting the number of hard and soft constraint violations into a fitness value that can be used by the Genetic Algorithm to compare chromosomes.

The implementation assigns a higher weight to hard constraint violations because satisfying hard constraints is more important than minimizing soft constraint violations.

The implemented weights are:

- `HARD_WEIGHT = 100`
- `SOFT_WEIGHT = 10`

The total penalty is calculated as:

$$\text{Penalty} = 100H + 10S$$

where H represents the number of hard constraint violations and S represents the number of soft constraint violations.

The fitness value is then calculated using:

$$\text{Fitness} = \frac{1}{1 + \text{Penalty}}$$

Therefore, a chromosome with fewer constraint violations receives a higher fitness value.

For example, if a timetable has:

$$H = 0, S = 0$$

then:

$$\text{Penalty} = 0$$

and:

$$\text{Fitness} = 1$$

Thus, **1.0 is the maximum fitness value** for the implemented fitness function.

The fitness calculation is used throughout the Genetic Algorithm to determine which chromosomes are better and should have a higher probability of being selected for reproduction.

```
src > main > java > ga > J FitnessFunction.java > FitnessFunction
1  package ga;
2
3  public class FitnessFunction {
4
5      private static final double HARD_WEIGHT = 100.0;
6      private static final double SOFT_WEIGHT = 10.0;
7
8      public static double calculateFitness(int hardViolations,
9          int softViolations) {
10
11          double penalty = HARD_WEIGHT * hardViolations
12              + SOFT_WEIGHT * softViolations;
13
14          return 1.0 / (1.0 + penalty);
15      }
16  }
```

Figure 6 Implementation of the fitness function

4.6 Selection

4.6.1 Tournament Selection

The project uses **Tournament Selection** to select chromosomes that will become parents for the next generation.

The selection process is implemented using a tournament size of **3**. In each tournament, three chromosomes are randomly selected from the current population. Their fitness values are then compared, and the chromosome with the highest fitness is selected as a parent.

The process can be summarized as follows:

1. Randomly select three chromosomes from the population.
2. Evaluate their fitness values.
3. Compare the fitness values.
4. Select the chromosome with the highest fitness.
5. Repeat the process whenever another parent is required.

Tournament Selection provides a balance between selection pressure and randomness. Chromosomes with better fitness have a higher chance of being selected, while weaker chromosomes can still participate because the tournament candidates are chosen randomly.

In the implementation, the tournament size is configurable through the `TournamentSelection` class, with a default value of **3**.

```
15  
16     @Override  
17     public Chromosome select(  
18         Population population,  
19         CourseGroup[] courseGroups,  
20         String[] preferredMorningInstructors) {  
21  
22         Chromosome best = null;  
23         double bestFitness = -1;  
24  
25         for (int i = 0; i < tournamentSize; i++) {  
26  
27             int randomIndex = RandomUtils.randomIndex(population.size());  
28  
29             Chromosome candidate = population.getChromosomes()[randomIndex];  
30  
31             double candidateFitness = ScheduleEvaluator.getFitness(  
32                 candidate,  
33                 courseGroups,  
34                 preferredMorningInstructors);  
35  
36             if (candidateFitness > bestFitness) {  
37  
38                 bestFitness = candidateFitness;  
39                 best = candidate;  
40             }  
41         }  
42  
43         return best;  
44     }  
45 }
```

Figure 7 Implementation of Tournament Selection

4.7 Crossover

4.7.1 Two-Point Crossover

The project uses **Two-Point Crossover** to create new chromosomes from two selected parent chromosomes.

The `TwoPointCrossover` class first checks whether crossover should be applied according to the configured crossover rate. If crossover is performed, two different crossover points are selected within the chromosome. The genes between these two points are then exchanged between the two parents.

For example:

Parent 1: [G1 G2 | G3 G4 G5 | G6 G7 G8]

Parent 2: [H1 H2 | H3 H4 H5 | H6 H7 H8]

After exchanging the middle segment:

Child 1: [G1 G2 | H3 H4 H5 | G6 G7 G8]

Child 2: [H1 H2 | G3 G4 G5 | H6 H7 H8]

In the scheduling problem, each gene represents a course assignment. Therefore, crossover allows an offspring to inherit groups of course assignments from both parent timetables.

The default crossover rate in the implementation is **80%**. When crossover is not applied, the selected parents are carried forward as offspring without exchanging their genes.

The crossover operation is performed before mutation and room repair in the Genetic Algorithm.

```

19 // Choose two different crossover points
20 int point1 = RandomUtils.randomIndex(length - 1) + 1;
21
22 int point2 = RandomUtils.randomIndex(length - 1) + 1;
23
24 while (point2 == point1) {
25     point2 = RandomUtils.randomIndex(length - 1) + 1;
26 }
27
28 // Make sure point1 is the smaller point
29 if (point1 > point2) {
30     int temp = point1;
31     point1 = point2;
32     point2 = temp;
33 }
34
35 // Create children
36 Gene[] childGenes1 = new Gene[length];
37 Gene[] childGenes2 = new Gene[length];
38
39 for (int i = 0; i < length; i++) {
40
41     if (i >= point1 && i < point2) {
42
43         childGenes1[i] = genes2[i];
44         childGenes2[i] = genes1[i];
45     } else {
46
47         childGenes1[i] = genes1[i];
48         childGenes2[i] = genes2[i];
49     }
50 }
51

```

Figure 8 Implementation of the Two-Point Crossover operator

4.8 Mutation

4.8.1 Random Reassignment Mutation

The project uses a **Random Reassignment Mutation** operator to introduce random changes into chromosomes and maintain diversity in the population.

The `RandomMutation` class randomly selects one gene from a chromosome. The selected gene keeps the same course, but its scheduling assignment is changed by randomly selecting:

- a new `TimeSlot`
- a new `Room`

For example:

Before mutation: **AI** → Sunday, 10:00-11:00, R101

After mutation: **AI** → Tuesday, 09:00-10:00, R102

The mutation operation is controlled by the mutation rate. In the default configuration, the mutation rate is 5%.

The mutation process helps introduce new scheduling combinations into the population and prevents the Genetic Algorithm from becoming too similar across generations.

After mutation, the resulting chromosome is passed to the **Room Repair** step to handle unsuitable room assignments.

The project also evaluates mutation rates of **1%, 5%, 10%, and 20%** as part of the experimental analysis.

```
src > main > java > ga > J RandomMutation.java > ...  
1  package ga;  
2  |  
3  import model.Chromosome;  
4  import model.Gene;  
5  import model.Room;  
6  import model.TimeSlot;  
7  import utils.RandomUtils;  
8  |  
9  public class RandomMutation implements Mutation {  
10 |  
11  @Override  
12  public Chromosome mutate(  
13      Chromosome chromosome,  
14      TimeSlot[] slots,  
15      Room[] rooms) {  
16  |  
17      Gene[] genes = chromosome.getGenes();  
18  |  
19      int geneIndex = RandomUtils.randomIndex(genes.length);  
20  |  
21      TimeSlot newTimeSlot = slots[RandomUtils.randomIndex(slots.length)];  
22  |  
23      Room newRoom = rooms[RandomUtils.randomIndex(rooms.length)];  
24  |  
25      Gene oldGene = genes[geneIndex];  
26  |  
27      genes[geneIndex] = new Gene(  
28          oldGene.getCourse(),  
29          newTimeSlot,  
30          newRoom);  
31  |  
32      return new Chromosome(genes);  
33  }  
34  }
```

Figure 9 Implementation of the Random Reassignment Mutation operator

4.9 Constraint Evaluation

The constraint evaluation is implemented in the `constraints` package. It is responsible for checking whether a timetable satisfies the required hard constraints and for calculating the soft constraint violations that affect the quality of the timetable.

The implementation separates the evaluation into hard constraints and soft constraints. The `ScheduleEvaluator` class combines the results of both types of constraints and provides the information required by the fitness function.

4.9.1 Hard Constraint Evaluation

The `HardConstraints` class checks the six hard constraints implemented in the project:

- **H1 – Instructor Conflict:** checks whether the same instructor is assigned to two courses at the same day and time.
- **H2 – Room Conflict:** checks whether the same room is assigned to two courses at the same day and time.
- **H3 – Student Conflict:** checks whether courses belonging to the same student group overlap.
- **H4 – Room Capacity:** checks whether the assigned room has enough capacity for the course.
- **H5 – Room Type:** checks whether the room type matches the course type.
- **H6 – Instructor Availability:** checks whether the instructor is available during the assigned time slot.

Each violation contributes to the total number of hard constraint violations.

4.9.2 Soft Constraint Evaluation

The `SoftConstraints` class evaluates the five soft constraints implemented in the system:

- **S1 – Avoid Early Slots**
- **S2 – Avoid Late Slots**
- **S3 – Minimize Student Gaps**
- **S4 – Instructor Preferences**
- **S5 – Minimize Working Days**

Unlike hard constraints, these violations do not make the timetable invalid. Instead, they increase the penalty and therefore reduce the fitness value.

4.9.3 Schedule Evaluation

The `ScheduleEvaluator` class combines the hard and soft constraint results for each chromosome.

The evaluation process can be summarized as:

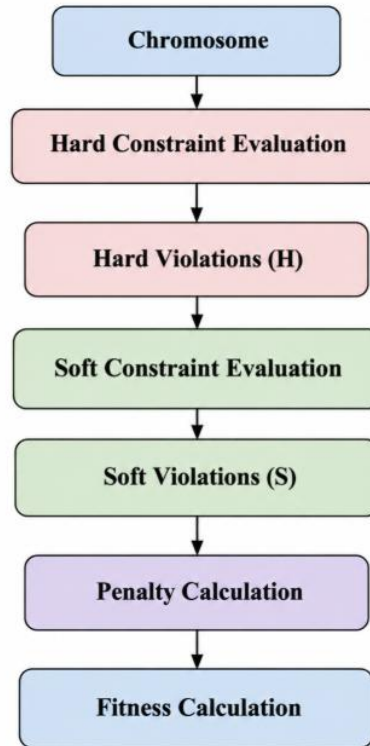


Figure 10 Schedule evaluation process from chromosome evaluation to fitness calculation

The resulting hard and soft violation counts are passed to the fitness calculation. Therefore, the constraint evaluation connects the timetable representation with the Genetic Algorithm's fitness function.

4.10 Genetic Algorithm Implementation

The `GeneticAlgorithm` class coordinates the main steps of the implemented Genetic Algorithm. It connects population initialization, fitness evaluation, parent selection, crossover, mutation, room repair, elitism, and generation replacement into one complete optimization process.

The algorithm starts by generating the initial population using the configured population size. Each chromosome is then evaluated according to its hard and soft constraint violations and corresponding fitness value.

For each generation, the algorithm performs the following operations:

1. Select parent chromosomes using Tournament Selection.
2. Apply Two-Point Crossover according to the crossover rate.
3. Apply Random Reassignment Mutation according to the mutation rate.
4. Apply Room Repair to improve unsuitable room assignments.
5. Preserve the best chromosomes using 5% elitism.
6. Form the new population.
7. Evaluate the new population.
8. Update the best solution found so far.

The algorithm keeps track of the best fitness value during the run. This allows the final solution to represent the best chromosome found throughout the entire execution rather than only the best chromosome of the last generation.

The implementation also records the best fitness after each generation. These values are later used to generate the convergence graph and analyze how the solution improves during the execution.

The algorithm terminates when the maximum number of generations is reached or when the best fitness does not improve for 50 consecutive generations. The number of generations actually used is recorded and reported with the final solution.

4.11 Convergence Graph

The `ConvergenceGraph` class is responsible for visualizing the improvement of the Genetic Algorithm during its execution.

During each generation, the `GeneticAlgorithm` records the best fitness value found so far. After the algorithm terminates, these recorded values are passed to the `ConvergenceGraph` class to generate a graph showing the relationship between the generation number and the best fitness value.

The graph uses:

- **X-axis:** Generation
- **Y-axis:** Best Fitness

The convergence graph helps determine how the quality of the solution changes throughout the execution. An increasing fitness curve indicates that the Genetic Algorithm is finding better solutions over successive generations. When the curve becomes stable, it indicates that the best solution is no longer improving significantly.

The graph is generated as an SVG file named `convergence_graph.svg`. The actual convergence graph and its results are presented and analyzed in the experimental results section.

5. Experimental Work

5.1 Experimental Setup

The experiments were conducted to evaluate the performance of the implemented Genetic Algorithm and to study the effect of different algorithm parameters and dataset sizes.

The experiments were implemented in Java using the same scheduling representation, constraint evaluation, fitness function, selection, crossover, mutation, room repair, and elitism mechanisms described in the previous sections.

The default configuration used for the main scheduling run was:

Parameter	Value
Population Size	100
Maximum Generations	500
Crossover Rate	80%
Mutation Rate	5%
Elitism Rate	5%
Tournament Size	3
Hard Constraint Weight	100
Soft Constraint Weight	10

The original dataset contains **10 courses**, while additional datasets containing **20, 30, 40, and 50 courses** were used to study the effect of increasing problem size.

The experiments investigated four main factors:

1. **Population Size:** 20, 50, 100, and 200.
2. **Mutation Rate:** 1%, 5%, 10%, and 20%.
3. **Crossover Rate:** 60%, 80%, and 90%.
4. **Dataset Size:** 10, 20, 30, 40, and 50 courses.

For each experiment, the main performance measures were the **best fitness, number of hard constraint violations, number of soft constraint violations, number of generations used, and runtime**.

The purpose of these experiments is to determine how changes in the Genetic Algorithm configuration and problem size affect the quality of the generated timetable and the computational cost of the algorithm.

5.2 Baseline Run

A baseline run was performed using the default Genetic Algorithm configuration. The purpose of this run was to verify that the implemented system can generate a complete timetable while satisfying the hard constraints and minimizing the soft constraints.

The baseline configuration used a population size of 100 chromosomes, a maximum of 500 generations, an 80% crossover rate, a 5% mutation rate, 5% elitism, and a tournament size of 3.

The final execution produced a complete timetable for all 10 courses. The resulting performance was:

Measure	Result
Courses Scheduled	10 / 10
Hard Violations	0
Soft Violations	0
Final Penalty	0.0
Best Fitness	1.0
Generations Used	56

The result shows that the Genetic Algorithm successfully generated a timetable with **zero hard constraint violations**. It also achieved zero soft constraint violations in this particular run. Consequently, the final penalty was zero and the fitness reached **1.0**, which is the maximum value of the implemented fitness function.

The algorithm terminated after 56 generations rather than reaching the maximum limit of 500 generations because the best solution stopped improving for the required number of consecutive generations. This demonstrates that the implemented early-stopping mechanism can reduce unnecessary generations after convergence.

```
===== FINAL TIMETABLE =====
Day      Time      Course      Instructor      Room
-----
Sunday   09:00-10:00  AI          Dr. Ahmad       R101
Tuesday  10:00-11:00  Database    Dr. Sara        R101
Tuesday  10:00-11:00  Networks    Dr. Omar        R102
Thursday 09:00-10:00  Operating Systems Dr. Ahmad       R101
Thursday 10:00-11:00  Software Engineering Dr. Lina        R101
Thursday 10:00-11:00  Algorithms   Dr. Khaled      R102
Thursday 09:00-10:00  Data Mining  Dr. Noor        R102
Tuesday  09:00-10:00  Computer Graphics Dr. Rami        LAB1
Monday   09:00-10:00  Web Programming Dr. Huda        LAB1
Sunday   10:00-11:00  Machine Learning Dr. Noor        R101

===== FINAL RESULTS =====
Courses scheduled: 10/10
Hard violations: 0
Soft violations: 0
Final penalty: 0.0
Best fitness: 1.0
Generations used: 56

Convergence graph generated:
convergence_graph.svg
```

Figure 11 Final timetable and performance results of the baseline run

5.3 Convergence Analysis

The convergence of the Genetic Algorithm was analyzed using the best fitness value recorded after each generation during the baseline run.

The convergence graph shows a rapid improvement in the quality of the best solution during the early generations. The fitness starts at a very low value and gradually improves during the first few generations. The best fitness reaches **1.0 at approximately generation 6**.

After reaching a fitness of 1.0, the curve remains constant until the end of the run. This indicates that the algorithm did not find a solution better than the obtained timetable, since **1.0 is the maximum possible fitness value** for the implemented fitness function.

The final solution has **zero hard constraint violations and zero soft constraint violations**, resulting in a penalty of **0.0** and a fitness of **1.0**.

Although the optimal solution was found early, the algorithm continued until generation **56** because the implemented termination mechanism also requires the best fitness to remain unchanged for 50 consecutive generations before early termination. Therefore, the run stopped after 56 generations rather than using the full maximum of 500 generations.

Overall, the convergence graph demonstrates that the algorithm was able to find an optimal timetable quickly and then maintain the best solution without losing it during subsequent generations.

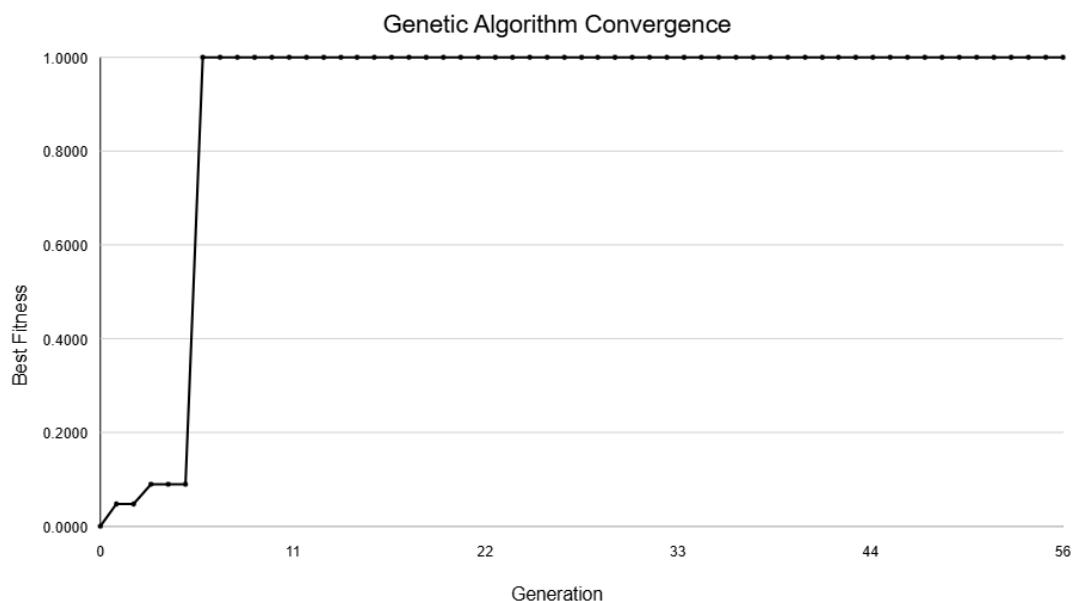


Figure 12 Convergence graph of the baseline Genetic Algorithm run

5.4 Population Size Experiment

The population size experiment shows a clear improvement in solution quality as the population size increases. The best fitness increased from **0.001560** with a population of 20 to **0.002770** with a population of 200. At the same time, the number of hard violations decreased from **3 to 1**, while soft violations decreased from **34 to 26**.

The number of generations used also decreased as the population became larger, from **352 generations** for population 20 to **248 generations** for population 200. This indicates that larger populations were able to find better solutions in fewer generations.

However, increasing the population size significantly increased the runtime. The runtime increased from **24,811 ms** for population 20 to **400,902 ms** for population 200. Therefore, although the population of 200 produced the best result among the tested configurations, it required substantially more computational time.

Overall, in this experiment, **population 200 produced the best timetable quality**, while **population 20 had the lowest runtime**. This demonstrates the trade-off between solution quality and computational cost when selecting the population size.

===== POPULATION SIZE EXPERIMENT =====					
Population	Generations	Hard	Soft	Best Fitness	Runtime(ms)
20	352	3	34	0.001560	24811
50	341	2	28	0.002079	69603
100	273	1	27	0.002695	142703
200	248	1	26	0.002770	400902

C:\Users\HP\Desktop\AI\Project\UniversityCourseScheduling>

Figure 13 Population size experiment results for different population sizes

5.5 Mutation Rate Experiment

The mutation rate experiment evaluates the effect of mutation rates of 1%, 5%, 10%, and 20% while keeping the other Genetic Algorithm parameters fixed.

The results show that increasing the mutation rate from 1% to 10% generally improved the solution quality. The best fitness increased from **0.001637** at 1% mutation to **0.003559** at 10%. The 10% mutation rate also achieved **0 hard violations**, which was the best hard-constraint result among the tested configurations.

The 20% mutation rate produced the lowest number of soft violations, with **25 soft violations**, compared with 28 for 10%, 30 for 5%, and 41 for 1%. However, its best fitness (**0.002849**) was lower than the fitness obtained with 10%.

The number of generations varied between the experiments. The 10% mutation rate required the most generations (**328**), while the 5% rate required **251** generations. The 20% rate reached its final result after **186 generations**.

Runtime also increased considerably for the 10% mutation rate, reaching **172,704 ms**. The 20% mutation rate required only **96,386 ms**, making it faster than both 5% and 10%.

Overall, **10% mutation produced the highest fitness and eliminated all hard violations**, making it the best configuration in terms of the primary solution-quality measure in this experiment. However, **20% mutation achieved the lowest number of soft violations and lower runtime**. This demonstrates the trade-off between solution quality, constraint satisfaction, and computational cost when selecting the mutation rate.

===== MUTATION RATE EXPERIMENT =====					
Mutation	Generations	Hard	Soft	Best Fitness	Runtime(ms)
1	174	2	41	0.001637	88439
5	251	1	30	0.002494	124812
10	328	0	28	0.003559	172704
20	186	1	25	0.002849	96386

c:\Users\HP\Desktop\AI\Project\UniversityCourseScheduling>

Figure 14 Mutation rate experiment results for different mutation rates

5.6 Crossover Rate Experiment

The crossover rate experiment evaluates the effect of crossover rates of **60%, 80%, and 90%** while keeping the other Genetic Algorithm parameters fixed.

The results show that both **60% and 80% crossover** achieved the same best fitness of **0.003984**, with **0 hard violations and 25 soft violations**. Therefore, both configurations produced solutions with the same measured quality in this run.

Although the 80% crossover rate achieved the same fitness as 60%, it required significantly more computational time. Its runtime was **321,068 ms**, compared with **177,803 ms** for 60%. It also required 406 generations compared with 307 generations for 60%.

The 90% crossover rate used the fewest generations, at **215 generations**, and had a runtime of **183,392 ms**. However, its solution quality was lower, with a fitness of **0.003115** and **32 soft violations**, although it still achieved **0 hard violations**.

Overall, the **60% crossover rate provided the best balance between solution quality and runtime** in this experiment. The 80% rate achieved the same fitness but required considerably more time, while the 90% rate was faster but produced a lower-quality solution.

===== Crossover Rate Experiment =====					
Crossover	Generations	Hard	Soft	Best Fitness	Runtime(ms)

60	307	0	25	0.003984	177803
80	406	0	25	0.003984	321068
90	215	0	32	0.003115	183392

Figure 15 Crossover rate experiment results for different crossover rates.

5.7 Dataset Size Experiment

The dataset size experiment evaluates how increasing the number of courses affects the performance of the Genetic Algorithm. The experiment was performed using datasets containing **10, 20, 30, 40, and 50 courses**, while keeping the other parameters fixed.

For the **10-course and 20-course datasets**, the algorithm achieved the optimal fitness of **1.0**, with **zero hard violations and zero soft violations**. This indicates that the algorithm was able to find completely satisfactory timetables for the smaller datasets.

As the dataset size increased to **30 courses**, the best fitness decreased to **0.016393**, with 0 hard violations and 6 soft violations. With **40 courses**, the fitness decreased further to **0.009901**, while the number of soft violations increased to 10.

For the largest dataset containing **50 courses**, the algorithm obtained a fitness of **0.002375**, with **1 hard violation and 32 soft violations**. This shows that the scheduling problem becomes more difficult as the number of courses increases because more assignments must be accommodated within the available time slots and rooms.

The runtime also increased substantially as the dataset became larger. It increased from only **561 ms** for 10 courses to **81,352 ms** for 50 courses. This demonstrates the increased computational cost associated with larger scheduling problems.

The number of generations generally increased from 53 generations for 10 courses to 232 generations for 40 courses. The 50-course experiment terminated after 144 generations, which can occur because the algorithm's termination condition also depends on the lack of improvement rather than only the maximum generation limit.

Overall, the results demonstrate that **smaller datasets are easier to schedule successfully**, while larger datasets produce more constraint violations and require considerably more computational time. This confirms the effect of problem size on the difficulty and computational cost of Genetic Algorithm-based course scheduling.

===== DATASET SIZE EXPERIMENT =====					
Courses	Generations	Hard	Soft	Best Fitness	Runtime(ms)
10	53	0	0	1.000000	561
20	116	0	0	1.000000	6197
30	158	0	6	0.016393	27039
40	232	0	10	0.009901	76506
50	144	1	32	0.002375	81352

Figure 16 Dataset size experiment results for different numbers of courses

5.8 Overall Experimental Results

The experimental results demonstrate how different Genetic Algorithm parameters and dataset sizes affect the quality and computational cost of the generated timetables. Four experiments were conducted: population size, mutation rate, crossover rate, and dataset size.

Summary of Experimental Results

Table 3 Summary of the overall experimental results

Experiment	Best Tested Configuration	Best Fitness	Hard Violations	Soft Violations	Runtime
Population Size	Population = 200	0.002770	1	26	400,902 ms
Mutation Rate	Mutation = 10%	0.003559	0	28	172,704 ms
Crossover Rate	Crossover = 60%	0.003984	0	25	177,803 ms
Dataset Size	10 / 20 courses	1.000000	0	0	561 / 6,197 ms

The population size experiment showed that larger populations generally produced better solutions. The population of 200 achieved the highest fitness within this experiment and reduced both hard and soft violations compared with smaller populations. However, this improvement came with a substantial increase in runtime.

The mutation rate experiment showed that the **10% mutation rate** achieved the highest fitness among the tested mutation rates and was the only configuration in this experiment to reach zero hard violations. The 20% mutation rate produced fewer soft violations, but its fitness was lower than that of the 10% configuration.

The crossover rate experiment showed that **60% and 80% crossover** produced the same fitness and constraint results in this run. However, the 60% configuration required considerably less runtime. The 90% crossover rate required fewer generations but produced more soft violations and a lower fitness.

The dataset size experiment demonstrated the strongest relationship between problem size and scheduling difficulty. The 10-course and 20-course datasets achieved the optimal fitness of **1.0** with no hard or soft violations. As the number of courses increased, the fitness decreased and the number of violations increased. The runtime also increased significantly, showing that larger scheduling problems require more computational effort.

Overall, the experiments demonstrate that there is a clear trade-off between **solution quality, constraint satisfaction, and computational cost**. Increasing the population size can improve the search quality but increases runtime, while mutation and crossover rates can affect the balance between exploration and the quality of the resulting timetable. Increasing the dataset size makes the scheduling problem more difficult and computationally expensive.

The experimental results also show that no single parameter value can be considered universally optimal. The best configuration depends on the dataset size and the desired balance between timetable quality and execution time.

5.9 Discussion of Results

The experimental results show that the Genetic Algorithm was able to generate feasible course timetables, but its performance depends on both the algorithm parameters and the size of the scheduling problem.

The **population size experiment** showed that increasing the population generally improved the quality of the obtained solution. The population of 200 achieved the highest fitness within this experiment and reduced the number of hard and soft violations compared with smaller populations. However, this improvement required a much higher runtime, demonstrating the trade-off between a larger search space and computational cost.

The **mutation rate experiment** showed that the tested mutation rates produced different results. The 10% mutation rate achieved the highest fitness and zero hard violations, while the 20% mutation rate achieved the lowest number of soft violations. This indicates that mutation rate affects the balance between introducing new scheduling combinations and maintaining good solutions.

The **crossover rate experiment** showed that the 60% and 80% rates achieved the same fitness and constraint results in the tested runs. However, the 60% rate required substantially less runtime. The 90% rate used fewer generations but resulted in more soft violations and lower fitness.

The **dataset size experiment** demonstrated that increasing the number of courses makes the scheduling problem more difficult. The 10-course and 20-course datasets achieved the optimal fitness of 1.0 with no violations. As the dataset increased to 30, 40, and 50 courses, the number of violations increased and fitness decreased. Runtime also increased considerably.

Overall, the experiments demonstrate that the Genetic Algorithm can successfully solve smaller scheduling instances and produce valid timetables. For larger datasets, finding high-quality solutions becomes more challenging because the available scheduling positions must accommodate a greater number of courses. Therefore, parameter selection should consider both the required timetable quality and the available computational resources.

6 Conclusion

This project presented an AI-based university course scheduling system using a Genetic Algorithm. The system was designed to automatically generate course timetables while considering different scheduling constraints related to instructors, rooms, students, and time slots.

The implemented Genetic Algorithm represents each timetable as a chromosome, where each gene contains a course, a time slot, and a room assignment. The algorithm starts with a randomly generated population and progressively improves the timetables through tournament selection, two-point crossover, random reassignment mutation, room repair, and elitism.

The fitness function combines hard and soft constraint violations using different penalty weights. This allows the algorithm to prioritize satisfying hard constraints while also improving the overall quality of the timetable by reducing soft constraint violations.

The baseline experiment demonstrated that the system can successfully generate a timetable containing all required courses with **zero hard violations and zero soft violations**, achieving the maximum fitness value of **1.0**.

The experimental analysis also showed that the performance of the Genetic Algorithm is affected by its configuration and by the size of the scheduling problem. Larger populations generally improved the quality of solutions but required more computational time. Different mutation and crossover rates produced different balances between solution quality and runtime. Increasing the number of courses made the scheduling problem more difficult and increased computational requirements.

Overall, the results demonstrate that a Genetic Algorithm can be effectively applied to university course scheduling and can produce high-quality timetables while handling multiple scheduling constraints.